Swap absorption



How _swap_absorption.py moves two-qubit gates next to routing SWAPs, and what changed in the CommutationChecker

Qiskit transpiler · written for readers who do not know Qiskit · October 2026

What this deck covers

1 Background

Circuits, qubit wires and gates

Why routing inserts SWAPs

The transpiler as a pipeline of passes

What it means for two gates to commute

2 The idea

Why a gate next to a SWAP is cheaper to compile

A worked example, before and after

The three rules every move obeys

3 The code

Pauli letters: the commutation model

Every class, function, record and constant in _swap_absorption.py

The planner's four stages traced on one running example, then the rewrite

4 The checker change

What CommutationChecker is

The defect in check_commutation_entries and its repair

The new Rust binding, its contract, files and tests

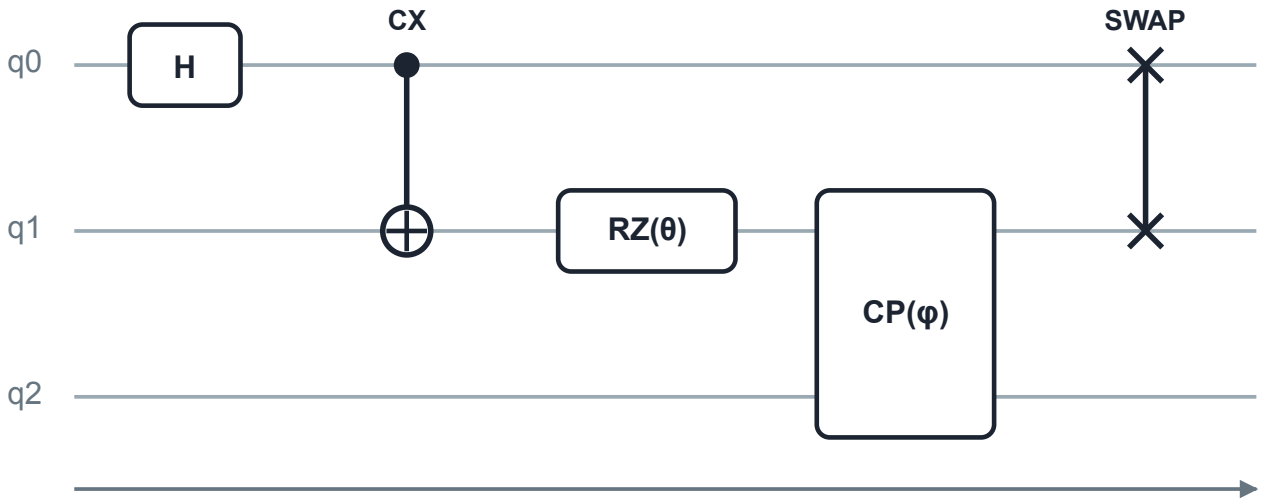
How to read it. Diagrams carry the explanation; the text around them names the exact functions and variables so you can open the source alongside. Monospace names such as `_find_moves` are identifiers in the code. Everything in the module is private (leading underscore) except the public `CommutationChecker.check_commutation_entries` method.

Part 1

Background: circuits, SWAPs, passes and commutation

A circuit is a timeline of gates on qubit wires

Reading a circuit diagram



time: gates are applied from left to right

- **A wire is one qubit**
  its state flows along the line
- **A box is a one-qubit gate**
  H, RZ(θ), RY, …
- **Dot and ⊕ joined: a CX gate**
  control on the dot, target on ⊕
- **A tall box is a two-qubit gate**
  CP(φ) acts on both wires
- **× — ×: a SWAP**
  exchanges the states of two qubits

Vocabulary used later

A gate's qubits are its arguments, written in order: CX(control, target).

A one-qubit gate like RZ(θ) takes an angle parameter θ.

In Qiskit

A QuantumCircuit is the user-facing object. The transpiler works on a DAGCircuit, a graph of the same gates.

Each gate becomes a DAGOpNode; node.qargs lists its qubits in order.

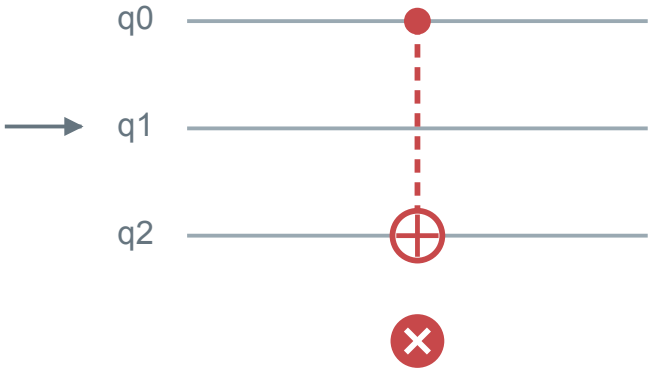
Hardware connectivity forces routing SWAPs

Hardware connectivity



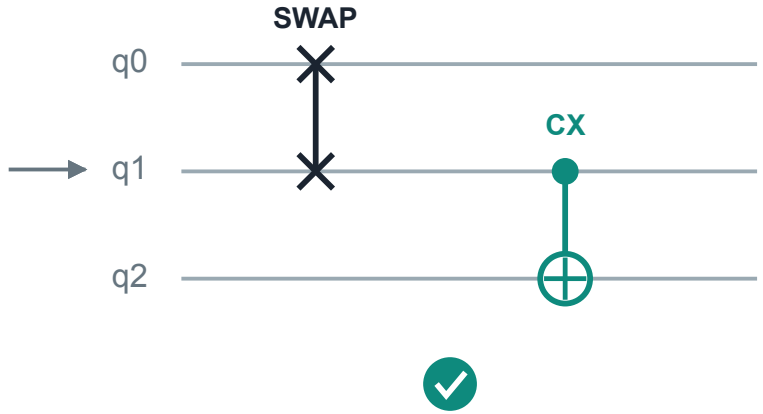
Two-qubit gates are allowed only between connected (neighbouring) qubits. Here q0 and q2 are not connected.

The program wants CX(q0, q2)



q0 and q2 are not neighbours, so this CX cannot run directly.

Routing inserts a SWAP



After SWAP(q0,q1) the data of q0 sits on q1, so CX(q1, q2) does the job.

Every SWAP costs about 3 CX gates on hardware. Routing adds many of them, so making each one cheaper matters.

A SWAP that routing inserted is just an ordinary standard swap gate in the circuit. The pass finds such gates by name; it does not care who inserted them.

The transpiler is a pipeline of passes



later: 2-qubit peephole synthesis merges SWAP + its new neighbour into one block

Enabled at optimization levels 2 and 3 by common.generate_translation_passmanager(). Levels 0 and 1 are unchanged.

A pass

A class with a run(dag) method that receives the circuit as a DAGCircuit and returns one.

_AbsorbIntoSwaps is a TransformationPass: it may rewrite gates but never changes the layout.

Why this spot in the pipeline

Routing has just produced the SWAPs. Translation has not yet decomposed anything, so the standard gate names are still visible.

The later 2-qubit peephole synthesis will merge each SWAP with its new neighbour.

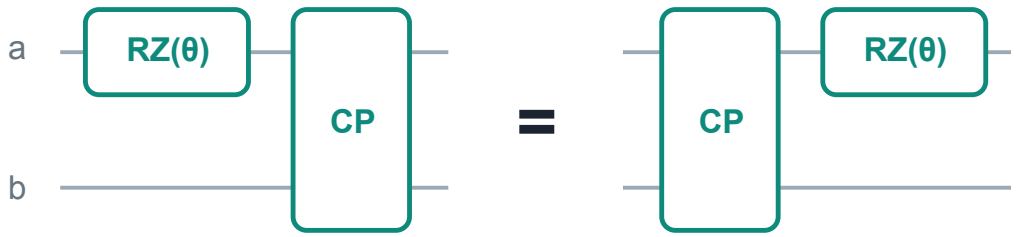
Where it is wired in

common.generate_translation_passmanager() adds the pass when optimization_level is 2 or 3.

Levels 0 and 1 and the default None are unchanged. Provider plugins calling the helper inherit the behaviour.

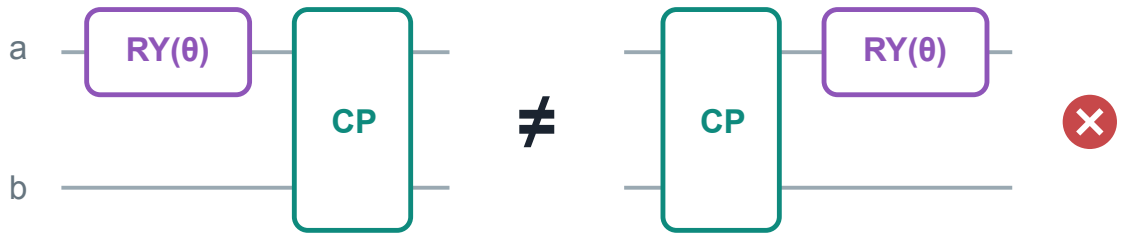
Commuting gates can change order safely

Commuting gates: order does not matter



Both act diagonally on wire a: RZ(θ)·CP = CP·RZ(θ) exactly.

Non-commuting gates: order matters



RY rotates around a different axis than CP's phase, so the two orders differ.

A gate may be moved past a neighbour only when the two commute: the circuit then computes exactly the same operation.

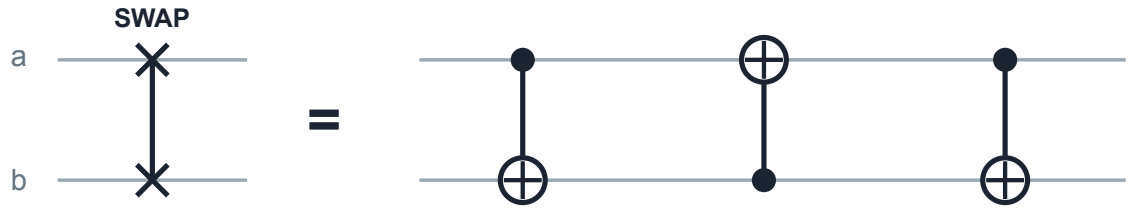
Checking commutation for every pair of gates with matrices would be slow and approximate.
The pass instead gives every gate one letter per wire and compares letters (next slides).

Exact commutation is what the pass needs: the rewritten circuit must compute precisely the same unitary, including its global phase, not merely something close to it.

Part 2

The idea: put a two-qubit gate right next to its SWAP

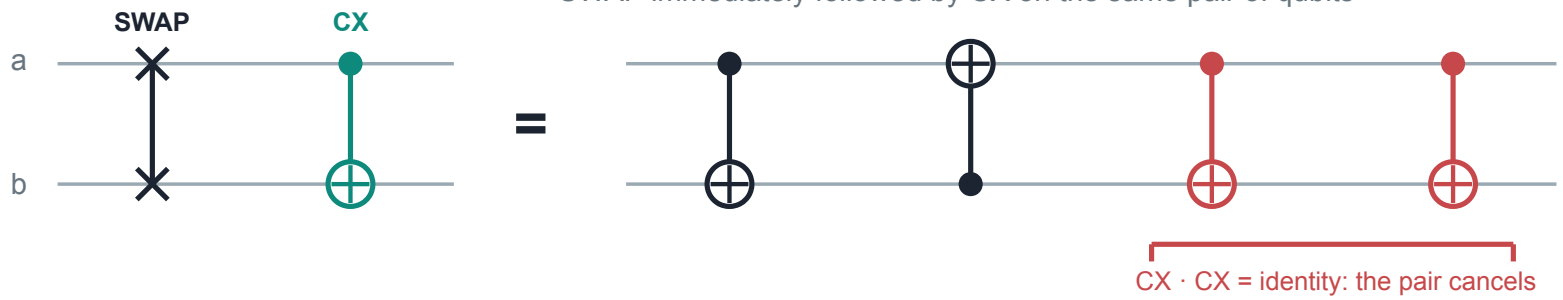
Why adjacency pays: SWAP + CX costs 2 CX instead of 4



A SWAP costs 3 CX gates

CX is the native entangling gate on many devices

SWAP immediately followed by CX on the same pair of qubits



→ 2 CX

instead of 3 + 1 = 4

In general

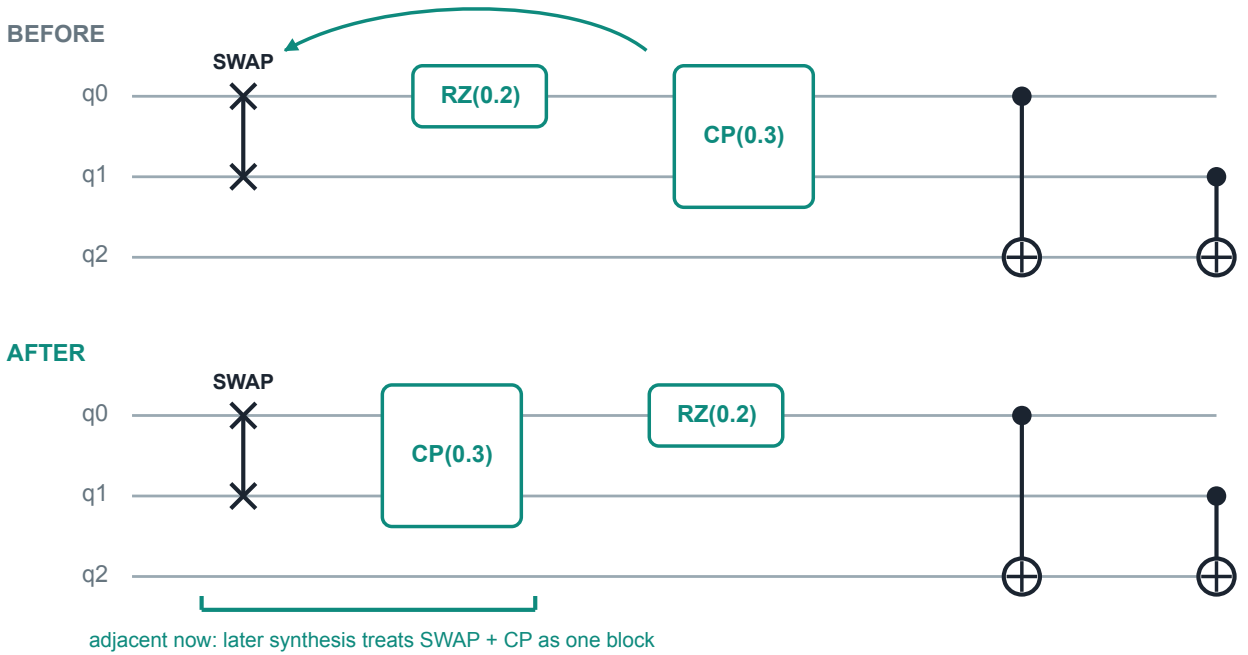
Any two-qubit gate adjacent to a SWAP on the same pair can be synthesized with it as one block. The pass only arranges adjacency; synthesis happens later.

References

[1] Liu, Li, Zhou. Not All SWAPs Have the Same Cost. arXiv:2205.10596

[2] Lao, Browne. 2QAN. arXiv:2108.02099

The pass in one example



On q0 only RZ stands between SWAP and CP. Both have label Z on that wire, so CP may be pulled left past RZ.

On q1 nothing is in the way.

Same gates, same parameters, same overall operation (checked to 1e-12, incl. global phase).

Only the order changed.

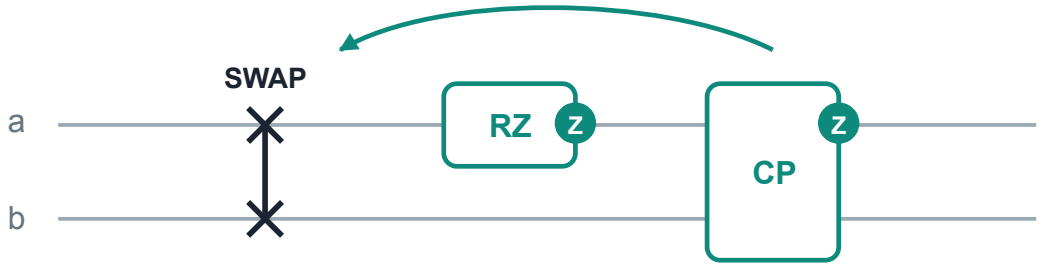
```
from qiskit import QuantumCircuit
from qiskit.converters import circuit_to_dag, dag_to_circuit
from qiskit.transpiler.preset_passmanagers._swap_absorption import _AbsorbIntoSwaps

qc = QuantumCircuit(3); qc.swap(0, 1); qc.rz(0.2, 0); qc.cp(0.3, 0, 1); qc.cx(0, 2); qc.cx(1, 2)
out = dag_to_circuit(_AbsorbIntoSwaps().run(circuit_to_dag(qc)))   # order now: swap, cp, rz, cx, cx  ·  Operator(out) == Operator(qc)
```

Replace RZ(0.2) with RY(1e-6) and nothing moves: Y and Z letters differ, however small the angle. Replace it with a barrier and nothing moves either.

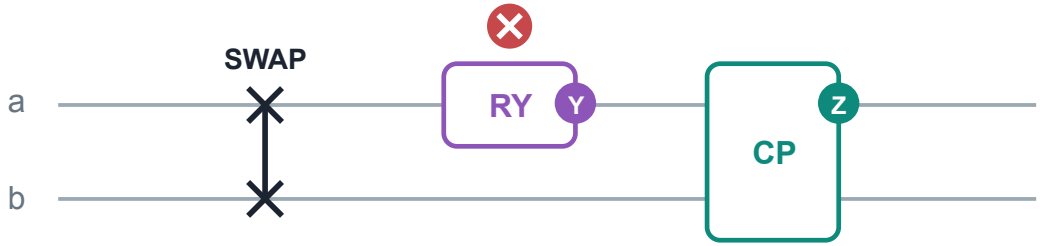
Three rules every move obeys

ALLOWED



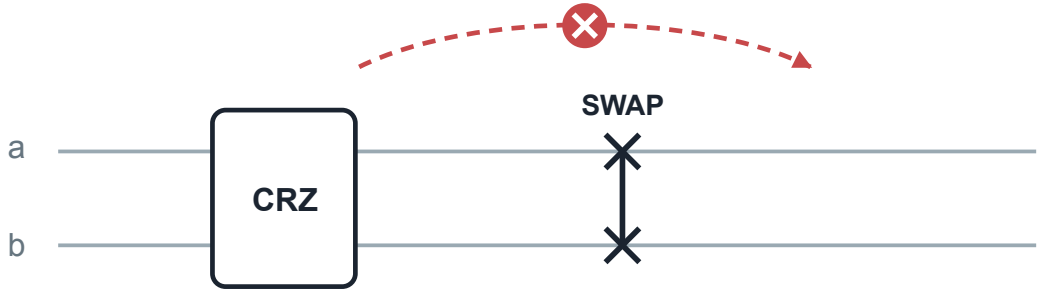
RZ has label Z on wire a and so does CP. Matching letters → CP slides left past RZ and lands right after the SWAP.

BLOCKED



RY has label Y, CP has label Z: a mismatch. CP stays where it is. Gates with no label ('-'), such as H, barrier or measure, block too.

NEVER ACROSS



A candidate keeps its side of the SWAP. Found before it → placed just before it. Found after it → placed just after it.

Only two-qubit gates on exactly the SWAP's pair of qubits are candidates. One-qubit gates can be crossed but are never moved themselves.

What the pass deliberately does not do

No SWAP is deleted

The pass keeps every SWAP and every gate with its parameters. It only reorders. Gate counts are unchanged until later synthesis merges blocks.

No synthesis, no layout change

It never builds matrices for the circuit, never picks routes, never changes the layout. Its whole output is better placement.

Never across a SWAP

A candidate may approach a SWAP but never jump over it. Asymmetric gates like CRZ(a,b) do not commute with SWAP(a,b).

No control flow

Control-flow blocks (if, while, for) are not entered. Barriers, measurements and resets are walls.

Conservative by design

RXX and RYY on the same pair commute, but their letters differ (XX vs YY), so no move. RY(0) keeps the Y letter: angles are never inspected.

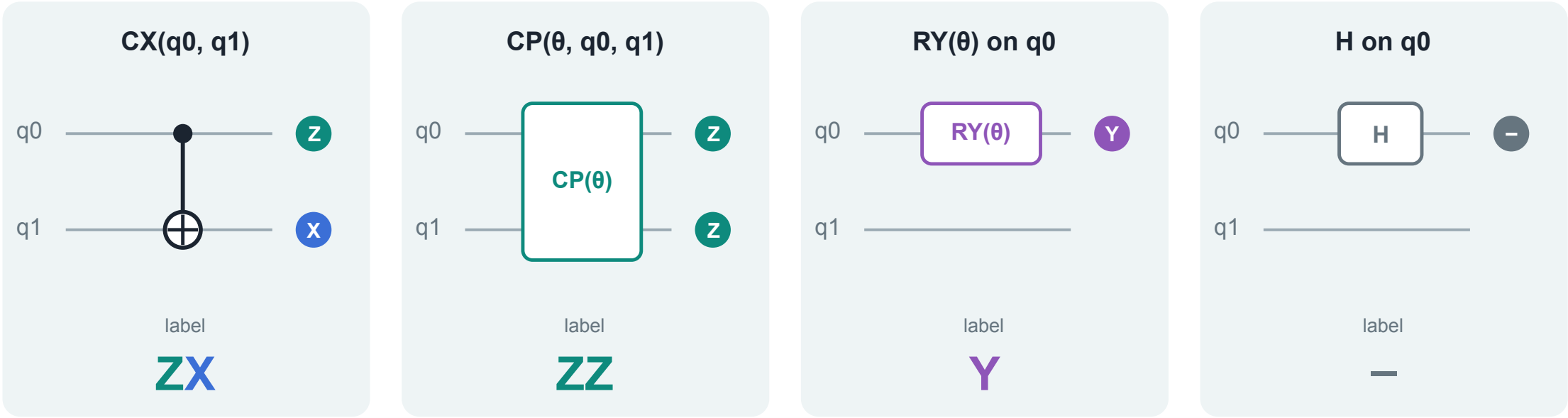
At most one new partner per SWAP

Each run picks at most one candidate for each SWAP that does not already touch a same-pair gate. Greedy, deterministic, no global search.

Part 3a

The commutation model: one Pauli letter per wire

One Pauli letter per wire



- **Z** commutes with Z on that wire: RZ, Z, S, T, P, CP, CZ, RZZ, …
- **Y** commutes with Y: RY, Y
- **X** commutes with X: RX, X, SX, the target of CX
- **–** no rule: H, barrier, measure, custom gates → impassable wall

Letters follow the gate's argument order: CX(control, target) → ZX. A letter is a certificate for moving, not a replacement gate.

The string follows node.qargs order, not Qiskit's Pauli-string convention. RZX(a, b) is 'ZX': Z on its first argument, X on its second.

Why matching letters guarantee exact commutation

1 Same letter on a shared wire

Both gates commute with that Pauli, so both are block diagonal in its eigenbasis on that wire.

Example: RZ and CP both commute with Z on wire a.

2 Fix every shared wire

Inside each eigenspace the two gates act only on their private qubits, which are disjoint.

Operations on disjoint qubits always commute.

3 Therefore they commute exactly

No tolerance, no approximation, global phase preserved.

The rewritten circuit is the same unitary.

The rule the planner applies: a candidate may cross a unit when, on every wire they share, the unit's letter equals the candidate's letter on that wire, and neither letter is '-'. Gates on wires the candidate does not touch are irrelevant.

Sufficient, not necessary

RXX and RYY on one pair commute, but XX and YY differ letter by letter, so the move is refused. That is accepted.

Angles are never inspected

RY(0) is the identity, yet it keeps letter Y and blocks a Z candidate. Parameter-independent rules are the point.

The 27 supported gate families

Standard gate names	Letters	Why
`rz, z, s, sdg, t, tdg, p, u1`	Z	Diagonal one-qubit gates: functions of Z
`rx, x, sx, sxdg`	X	Functions of X
`ry, y`	Y	Functions of Y
`cz, cp, crz, cu1, cs, csdg, rzz`	ZZ	Diagonal two-qubit gates
`cx, crx, csx, rzx`	ZX	Z on the first argument, X on the second
`rxx`	XX	X on both arguments
`ryy`	YY	Y on both arguments

Extra entries in the commutation table do not enlarge this set: the allowlist is checked before any lookup, so new table coverage can never enable new moves by accident.

Two extra ZZ forms

A genuine two-qubit UnitaryGate whose matrix has every off-diagonal entry exactly zero (1e-13 is enough to reject).

A recognised CX(a,b); D(b); CX(a,b) block with D a Z-letter gate, e.g. CX; RZ; CX = RZZ.

Only genuine standard gates

node.is_standard_gate() must be true and the name must be in the allowlist.

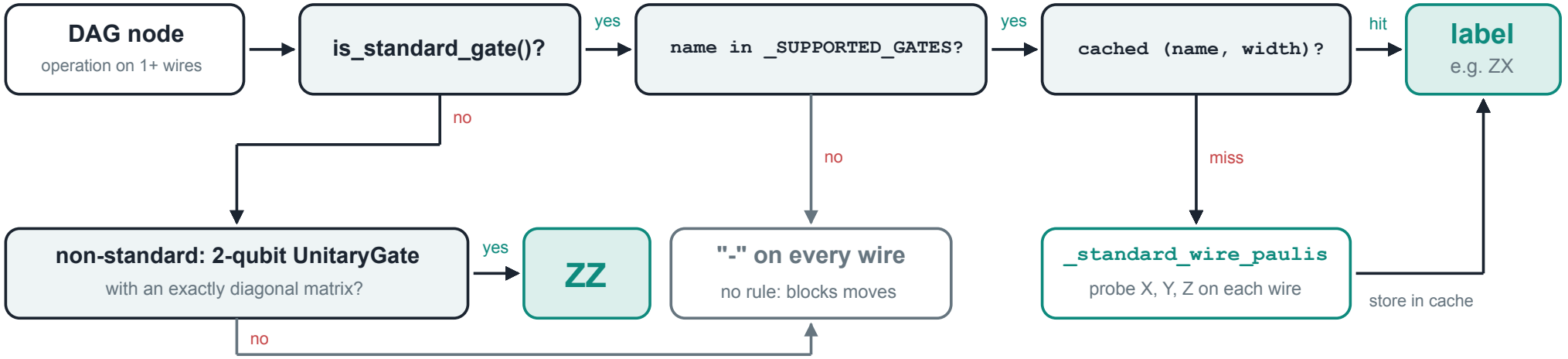
A custom Gate named 'cx', or an open-control CX, gets '-'.

Movable versus crossable

Two-qubit entries can be moved next to a SWAP.

One-qubit entries are only ever crossed.

How a node gets its letters: _wire_paulis



Only genuine standard gates from the fixed allowlist of 27 names ever reach the lookup.

A custom gate merely named 'cx' or 'rz', an open-control CX, a barrier or a measurement all get '-'.

The cache lives for one run of the pass and is keyed by (name, width), so angles and qubit numbers never matter.

```
def _wire_paulis(node: DAGOpNode, cache: dict | None = None) -> str:
    # returns one letter per qubit of node; '-' means no rule
    # standard gate -> allowlist -> cache[(name, width)] -> _standard_wire_paulis(op, width)
    # UnitaryGate, 2 qubits, exactly diagonal -> 'ZZ'      anything else -> '-' * width
```

The three constants and the probe: _standard_wire_paulis

CP(θ, a, b)
the gate in the circuit

→

representative: CZ
a proof stand-in, never inserted

_LOOKUP_REPRESENTATIVES

```
rx → X      ry → Y      rz → Z
p, u1 → Z   (phase gates are functions of Z)
cp, cu1 → CZ   (CP(θ) is a function of CZ)
every other allowed gate → itself
```

probe on local wire	X	Y	Z	label
wire 0 (a)	False	False	**True**	Z
wire 1 (b)	False	False	**True**	Z

result: **ZZ**

`check_commutation_entries(CZ, (0, 1), pauli, (wire,)) is True`

Exactly one True per wire → that letter; zero or two Trues → '-'. The raw table holds no CP-vs-Pauli entry at all (None), which is why the parameter-free CZ stands in. Nothing about θ is inspected, so RZ(0), RZ(1e-6) and RZ(θ) all behave alike.

_SUPPORTED_GATES

frozenset of the 27 names. Checked before any lookup, so table coverage alone can never admit a new gate.

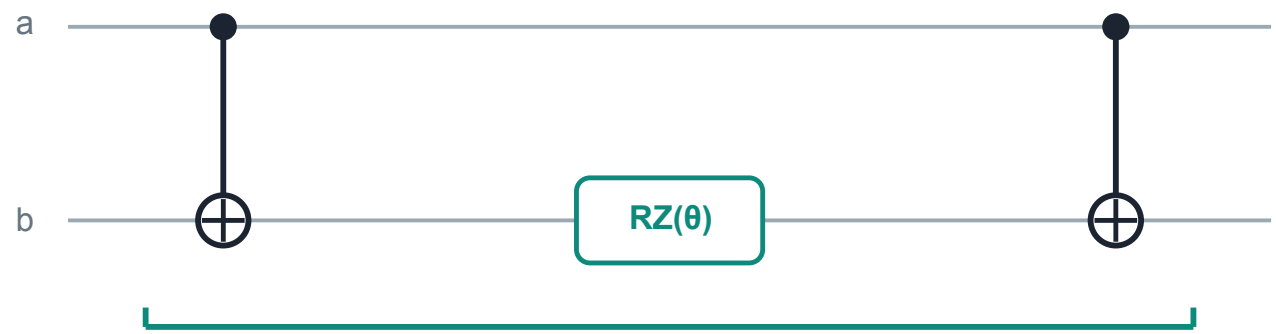
_PAULI_PROBES

(('X', XGate()), ('Y', YGate()), ('Z', ZGate())): the three questions asked on every local wire.

_LOOKUP_REPRESENTATIVES

rx→X, ry→Y, rz→Z, p/u1→Z, cp/cu1→CZ. Parameter-free stand-ins for the proof query; never inserted into the circuit.

The compound unit: CX, D, CX counts as one ZZ gate



= RZZ(θ)

a diagonal two-qubit gate

the 3 original gates move together

Recognised only when: both CX are standard gates with the same control a and target b;

the control wire a has nothing between them; exactly one Z-labelled 1-qubit gate (RZ, S, T, P, …) sits on b between them.

Exact conditions checked in the code

First CX: standard, letters ZX. Next node on the target wire: a one-qubit node with letter Z. Next node after that on the target: a standard CX with the same (control, target). The control wire's next node after the first CX must be that same closing CX.

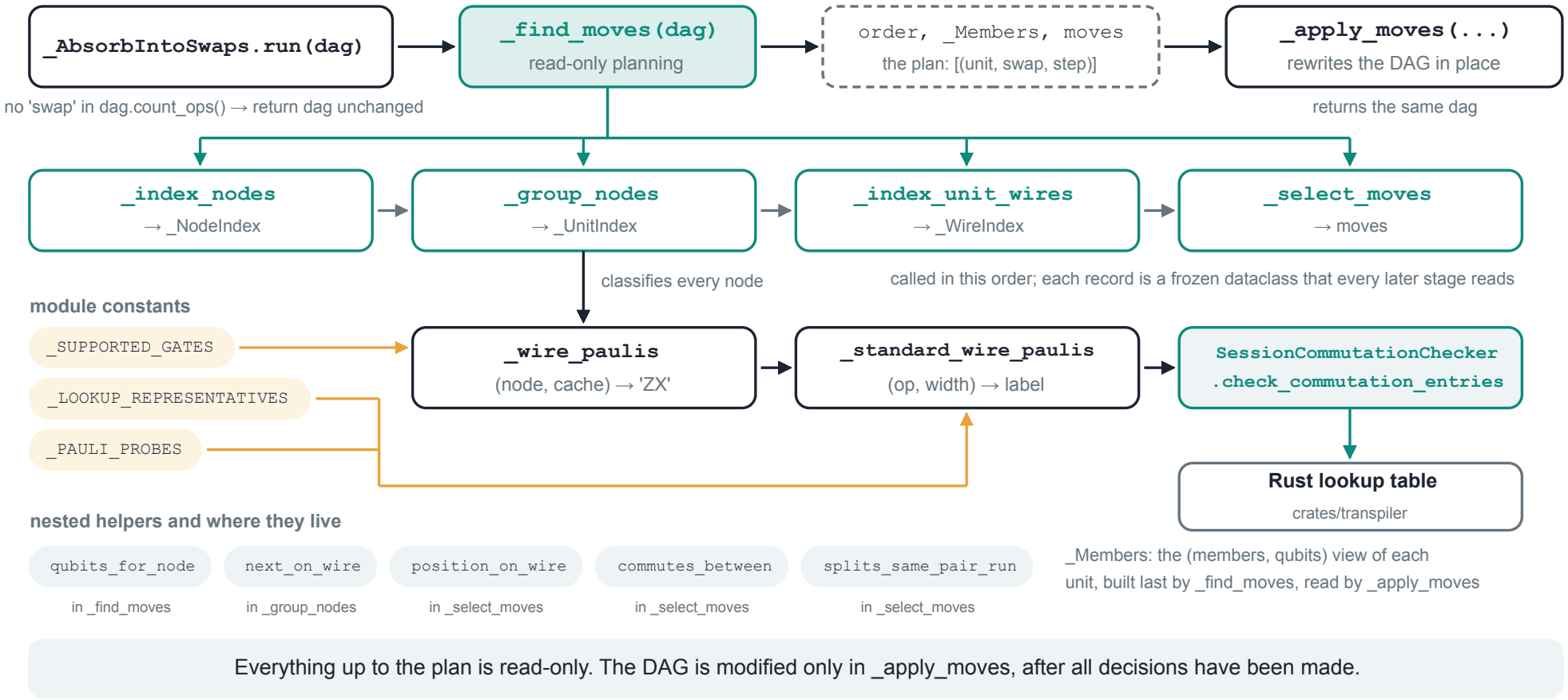
Why it matters

Routed circuits from Hamiltonian simulation and QAOA are full of CX-RZ-CX blocks. Treating them as one diagonal unit lets the pass pull a whole block next to a SWAP instead of only its outer CX.

Part 3b

Inside _swap_absorption.py: every class and function

Module map: who calls whom



Three public-facing facts for newcomers: the pass has no constructor options; it never recurses into control-flow blocks; it mutates and returns the DAG it was given.

_AbsorbIntoSwaps.run: the entry point

```
class _AbsorbIntoSwaps(TransformationPass):
    """Move a two-qubit interaction next to a ``swap``
    on the same qubit pair."""

    def run(self, dag: DAGCircuit) -> DAGCircuit:
        if "swap" not in dag.count_ops(recurse=False):
            return dag
        order, units, moves = _find_moves(dag)
        return (_apply_moves(dag, order, units, moves)
                if moves else dag)
```

For readers new to Qiskit passes

- TransformationPass: a pass that may change the circuit but must keep the layout. The pass manager wires run(dag) into the pipeline.
- DAGCircuit: Qiskit's graph form of a circuit. dag.count_ops() counts gates by name; topological_op_nodes() lists gates in dependency order.
- No state is kept between runs: the only cache is the letter cache created inside _group_nodes for one call.

1 Fast exit

No operation named swap at the top level → return the DAG untouched. Control-flow bodies are not searched (recurse=False).

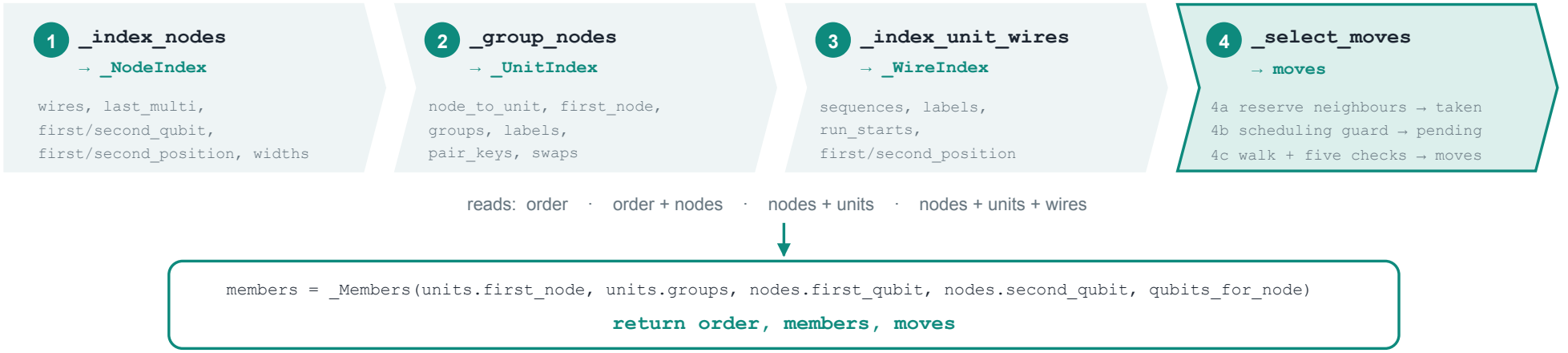
2 Plan

_find_moves(dag) returns the topological node list, a _Members view of the units, and the list of (unit, swap, step) moves. Read-only.

3 Rewrite

_apply_moves removes the chosen units and re-inserts each one glued to its SWAP. Returns the same DAG object.

_find_moves: four stages and three records

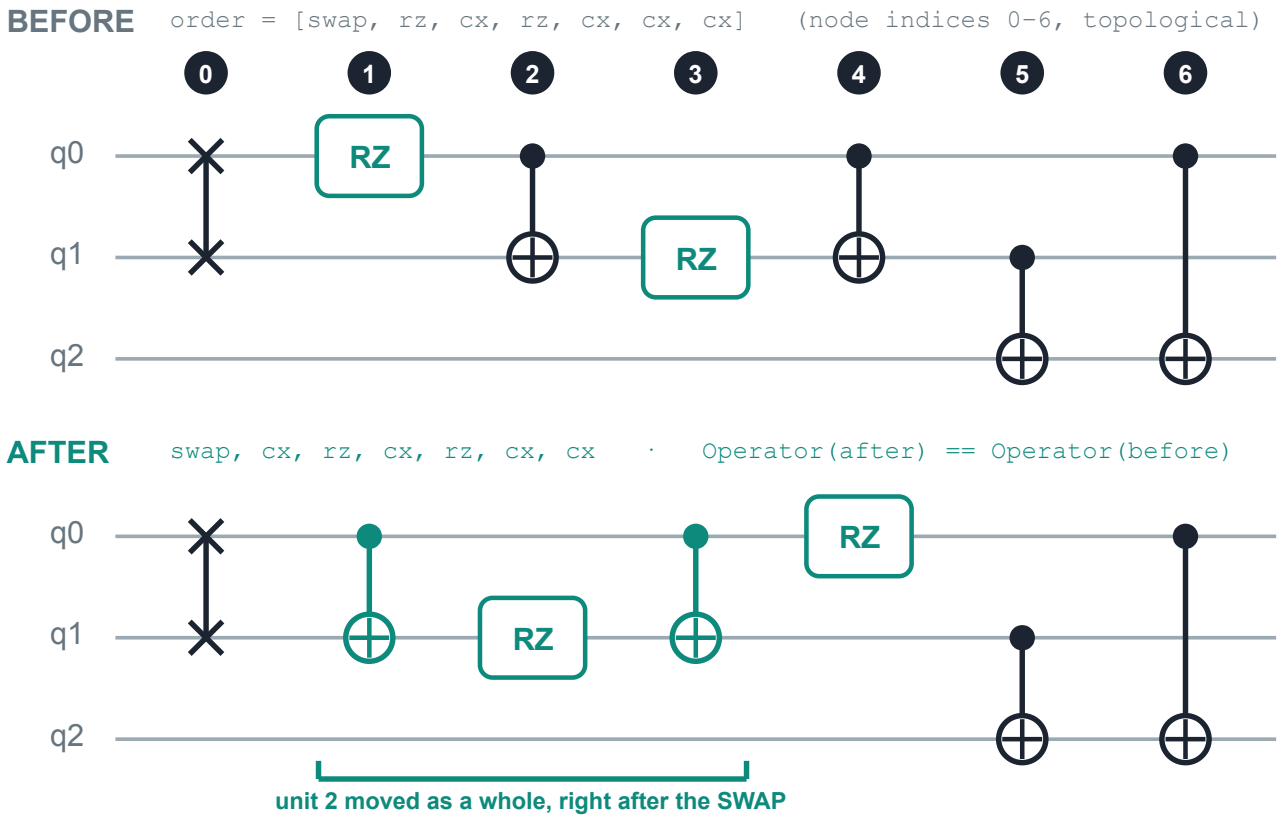


Stages 1–3 only read the circuit and build records; stage 4 picks the moves. The DAG is untouched until _apply_moves.

```
def _find_moves(dag: DAGCircuit):
    order = list(dag.topological_op_nodes())
    qubit_index = {qubit: index for index, qubit in enumerate(dag.qubits)}
    nodes = _index_nodes(order, qubit_index, dag.num_qubits())    # stage 1
    units = _group_nodes(order, nodes)                             # stage 2
    wires = _index_unit_wires(nodes, units)                        # stage 3
    moves = _select_moves(nodes, units, wires)                     # stage 4
    def qubits_for_node(node):
        return [qubit_index[qubit] for qubit in order[node].qargs]
    members = _Members(units.first_node, units.groups,
                       nodes.first_qubit, nodes.second_qubit, qubits_for_node)
    return order, members, moves
```

Return value	Meaning
`order`	list[DAGOpNode] in topological order; every node index refers to it
`members`	_Members: units[u] gives (member node indices, qubit index set)
`moves`	list of (unit, swap, step): step = +1 found after the SWAP, −1 before

The running example used on every stage slide



Seven nodes, five units, one move

`node 0`
swap(q0, q1): the routing SWAP

`node 1`
rz(q0): one-qubit, letter Z, crossable

`nodes 2–4`
cx(q0,q1); rz(q1); cx(q0,q1): one ZZ unit

`node 5`
cx(q1, q2): keeps q1 busy after the SWAP

`node 6`
cx(q0, q2): keeps q0 busy after the SWAP

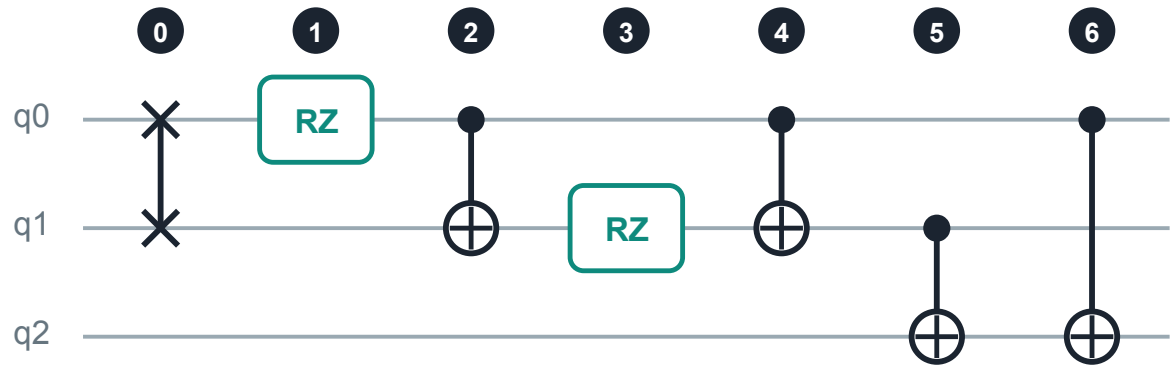
`moves = [(2, 0, +1)]`

unit 2 is found after SWAP unit 0 (step +1) and is placed right after it. Every stage slide shows the arrays this circuit produces.

```
qc = QuantumCircuit(3); qc.swap(0, 1); qc.rz(0.2, 0); qc.cx(0, 1); qc.rz(0.5, 1); qc.cx(0, 1); qc.cx(1, 2); qc.cx(0, 2)
order, members, moves = _find_moves(circuit_to_dag(qc))     # moves == [(2, 0, 1)]
```

Stage 1 · _index_nodes: what it builds

`_index_nodes(order, qubit_index, num_wires) → _NodeIndex`



numbers: index in order (topological)

`wires[0]`	`[0, 1, 2, 4, 6]`	nodes touching q0, in order
`wires[1]`	`[0, 2, 3, 4, 5]`	
`wires[2]`	`[5, 6]`	
`last_multi`	`[4, 4, 1]`	position of the last node with ≥ 2 qubits

A position on a wire is an index into that wire's list, not into order.
last_multi stays −1 on a wire with no multi-qubit node.

node	0	1	2	3	4	5	6
op	swap	rz	cx	rz	cx	cx	cx
qargs	(0,1)	(0)	(0,1)	(1)	(0,1)	(1,2)	(0,2)
widths	2	1	2	1	2	2	2
first_qubit	0	0	0	1	0	1	0
second_qubit	1	−1	1	−1	1	2	2
first_position	0	1	2	2	3	4	4
second_position	0	−1	1	−1	3	0	1

per node: how many qubits, which wires, and the index on each of those wires
−1 = not applicable: a one-qubit node has no second qubit or second position

Node 2 (cx) sits at index 2 of wires[0] and index 1 of wires[1]; node 6 is the last multi-qubit node on q0 and on q2, so last_multi[0] = last_multi[2] = its positions there.

Positions are per wire. Node 3 (rz on q1) has first_position 2 because two nodes, 0 and 2, precede it on wires[1]; its second_qubit and second_position stay −1.

Stage 1 · _index_nodes: how it works

```
def _index_nodes(order, qubit_index, num_wires: int) -> _NodeIndex:
    wires = [[] for _ in range(num_wires)]
    last_multi = [-1] * num_wires
    ...  # first_qubit, second_qubit, first_position, second_position:
    ...  # [-1] per node; widths: [0] per node
    for node, op_node in enumerate(order):
        qubits = op_node.qargs
        width = len(qubits)
        widths[node] = width
        if width == 1:
            qubit = qubit_index[qubits[0]]
            wire = wires[qubit]
            first_qubit[node] = qubit
            first_position[node] = len(wire)
            wire.append(node)
        elif width == 2:
            qubit_a = qubit_index[qubits[0]]
            qubit_b = qubit_index[qubits[1]]
            wire_a = wires[qubit_a]
            wire_b = wires[qubit_b]
            first_qubit[node] = qubit_a
            second_qubit[node] = qubit_b
            first_position[node] = last_multi[qubit_a] = len(wire_a)
            second_position[node] = last_multi[qubit_b] = len(wire_b)
            wire_a.append(node)
            wire_b.append(node)
        else:
            for qubit in qubits:
                wire = wires[qubit_index[qubit]]
                if width > 1:
                    last_multi[qubit_index[qubit]] = len(wire)
                wire.append(node)
    return _NodeIndex(wires, last_multi, first_qubit, second_qubit,
                      first_position, second_position, widths)
```

Step by step

1. One pass over order
Node i is visited with its qargs. qubit_index maps each Qubit object to its integer wire number, so everything after this works on small integers.

2. Width 1: one wire
The qubit goes into first_qubit[i]; the position is the wire's current length, stored in first_position[i] before the node is appended to wires[q].

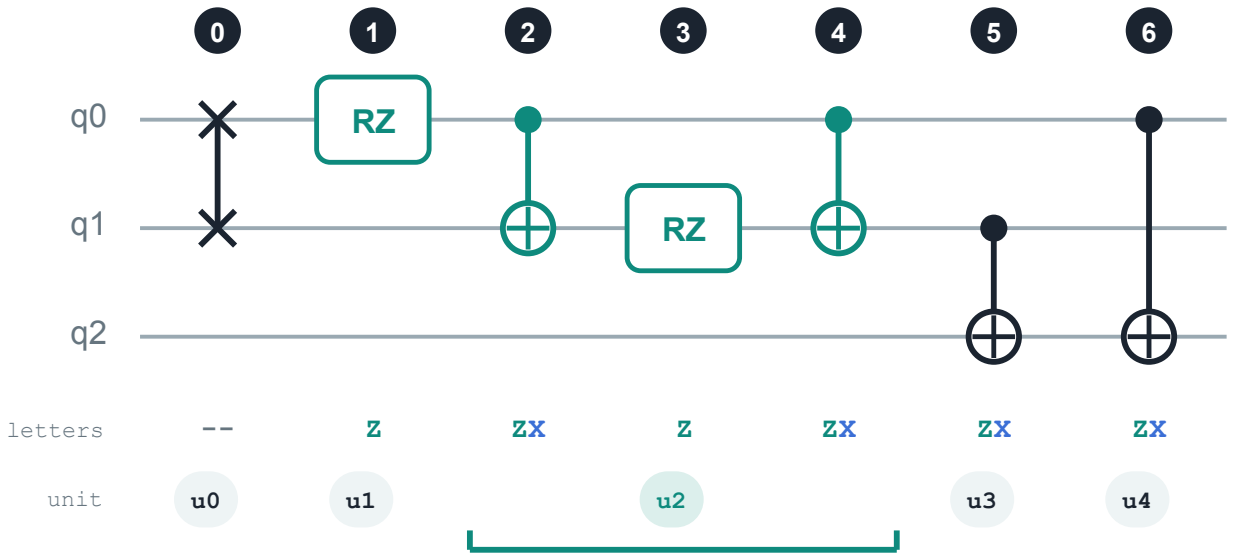
3. Width 2: two wires and last_multi
Both qubits and both positions are stored in argument order (first = qargs[0]). The same assignment updates last_multi on both wires, so after the loop it holds the position of the last multi-qubit node on each wire.

4. Any other width
A barrier on three qubits, a measure, a reset or a control-flow block is appended to every wire it touches and bumps last_multi when it has more than one qubit, but gets no first/second entries: they stay −1 and the node can never be a candidate.

5. What this buys
Every later question is an O(1) lookup: 'what follows this node on wire q' is wires[q][position + 1], and 'is there multi-qubit work after the SWAP' is a comparison with last_multi[q].

Stage 2 · _group_nodes: what it builds

`_group_nodes(order, nodes) → _UnitIndex`



`node_to_unit`	`[0, 1, 2, 2, 2, 3, 4]`
`first_node`	`[0, 1, 2, 5, 6]`
`groups`	`{2: [2, 3, 4]}`
`labels`	`['--','Z','ZZ','ZX','ZX']`
`pair_keys`	`[1, -2, 1, 5, 2]`
`swaps`	`[0]`

node → unit · unit → its first node · members, only for the group · letters in first-node qubit order · pair key or −2 / −1 · standard SWAP units

pair_keys: one integer per unit

`two-qubit unit: min(a,b)·num_wires + max(a,b)`

`(0,1) → 0·3+1 = 1` `(0,2) → 0·3+2 = 2` `(1,2) → 1·3+2 = 5`

`one-qubit unit → _SINGLE_QUBIT_UNIT = -2;  anything else → _OTHER_UNIT = -1`

Equal pair keys mean the same unordered pair of wires: SWAP and candidate are matched in O(1).

the CX·D·CX test at node 2 (a standard cx with letters ZX)

- next on target q1 after node 2: node 3, one-qubit, letter Z
- next on q1 after node 3: node 4, a standard cx on (0, 1)
- next on control q0 after node 2 is node 4 too: nothing between

`groups[2] = [2, 3, 4]; node_to_unit[3] = node_to_unit[4] = 2`

`labels[2] = 'ZZ'  (CX · Z-diagonal · CX is a function of ZZ)`

nodes 3 and 4 are skipped when the loop reaches them

Letters are per node here; stage 3 splits a unit's two letters between its two wires. The pair key ignores argument order, so cx(1, 0) and swap(0, 1) share key 1.

Stage 2 · _group_nodes: how it works

```
def _group_nodes(order, nodes: _NodeIndex) -> _UnitIndex:
    cache = {}
    ...  # next_on_wire(qubit, position) -> the node after that position, or None
    ...  # node_to_unit = [-1] * len(order); the five output collections start empty
    for node, op_node in enumerate(order):
        if node_to_unit[node] >= 0:
            continue
        width = nodes.widths[node]
        label = _wire_paulis(op_node, cache)
        unit = len(first_node)
        if label == "ZX" and op_node.name == "cx" and op_node.is_standard_gate():
            control, target = nodes.first_qubit[node], nodes.second_qubit[node]
            middle = next_on_wire(target, nodes.second_position[node])
            if (middle is not None
                and _wire_paulis(order[middle], cache) == "Z"
                and (last := next_on_wire(target, nodes.first_position[middle])) is not None
                and order[last].name == "cx" and order[last].is_standard_gate()
                and nodes.first_qubit[last] == control
                and nodes.second_qubit[last] == target
                and nodes.widths[last] == 2
                and next_on_wire(control, nodes.first_position[node]) == last):
                groups[unit] = [node, middle, last]
                node_to_unit[middle] = node_to_unit[last] = unit
                label = "ZZ"
        node_to_unit[node] = unit
        first_node.append(node)
        labels.append(label)
        if width == 2:
            qubit_a, qubit_b = nodes.first_qubit[node], nodes.second_qubit[node]
            pair_keys.append(qubit_a * num_wires + qubit_b if qubit_a < qubit_b
                             else qubit_b * num_wires + qubit_a)
            if op_node.name == "swap" and op_node.is_standard_gate():
                swaps.append(unit)
        else:
            pair_keys.append(_SINGLE_QUBIT_UNIT if width == 1 else _OTHER_UNIT)
    return _UnitIndex(node_to_unit, first_node, groups, labels, pair_keys, swaps)
```

Step by step

1. Skip members of an earlier group
node_to_unit starts at −1. A node the CX·D·CX test already claimed (nodes 3 and 4 here) is skipped and never becomes a unit of its own.

2. Classify through the cache
_wire_paulis(node, cache) returns the letters; the cache maps (name, width) to them, so the checker is consulted once per gate kind per run.

3. The CX·D·CX test
Only a standard cx with letters ZX starts it. Next on the target: a one-qubit Z node; next after it on the target: a standard cx with the same control and target; next on the control after the first cx: that same closing cx. On success the three nodes share the unit, groups records them, and the letters become ZZ.

4. Record the unit
first_node, labels and pair_keys get one entry per unit: key min·num_wires + max for two-qubit units, −2 for one-qubit units, −1 otherwise. A genuine standard swap also joins swaps.

5. Why units
From here on the planner reasons about units: a candidate is a unit, a wall is a unit, and a group's three nodes move together.

Stage 3 · _index_unit_wires: what it builds

`_index_unit_wires(nodes, units) → _WireIndex`

	position	0	1	2	3
q0	sequences	u0 swap	u1 rz	u2 cx·rz·cx	u4 cx(0,2)
	labels	–	Z	Z	Z
	run_starts	0	1	1	1

u2 is 'ZZ' and q0 is its first qubit → labels[0] = 'Z' · u4 = cx(0,2) is 'ZX' → 'Z' here

	position	0	1
q2	sequences	u3 cx(1,2)	u4 cx(0,2)
	labels	X	X
	run_starts	0	0

both CX gates target q2 → 'X'; one run from 0

	position	0	1	2
q1	sequences	u0 swap	u2 cx·rz·cx	u3 cx(1,2)
	labels	–	Z	Z
	run_starts	0	1	1

u2 on its second qubit → labels[1] = 'Z' · u3 = cx(1,2): q1 is its first qubit → 'Z'

first_position	[0, -1, 2, 2, 3]
second_position	[0, -1, 1, 0, 1]

per unit: its position on the wire of its first / second qubit; −1 for u1, which is a one-qubit unit.
wires[0] = [0, 1, 2, 4, 6] maps to units [0, 1, 2, 2, 4]: the repeat is merged, so u2 takes one position per wire.
run_starts[q][p] = where the run of equal letters that contains position p began: 0, 1, 1, 1 on q0.

Positions here count merged units, not nodes. The SWAP's own letter is '-', so it starts a run of its own.

Stage 3 · _index_unit_wires: how it works

```
def _index_unit_wires(nodes: _NodeIndex, units: _UnitIndex) -> _WireIndex:
    num_units = len(units.first_node)
    first_position = [-1] * num_units
    second_position = [-1] * num_units
    ...  # sequences, wire_labels, run_starts = [], [], []
    for qubit, wire in enumerate(nodes.wires):
        ...  # sequence, labels, starts = [], [], []
        previous_unit = -1
        for node in wire:
            unit = units.node_to_unit[node]
            if unit == previous_unit:
                continue
            previous_unit = unit
            position = len(sequence)
            label = units.labels[unit]
            head = units.first_node[unit]
            if units.pair_keys[unit] >= 0:
                if nodes.first_qubit[head] == qubit:
                    first_position[unit] = position
                    label = label[0]
                else:
                    second_position[unit] = position
                    label = label[1]
            else:
                label = label[0] if label else "-"
            starts.append(starts[-1] if position and labels[-1] == label
                          else position)
            sequence.append(unit)
            labels.append(label)
        sequences.append(sequence)
        wire_labels.append(labels)
        run_starts.append(starts)
    return _WireIndex(sequences, wire_labels, run_starts,
                      first_position, second_position)
```

Step by step

1. Walk each wire, merging repeats
The node list of a wire is mapped node by node to units. A unit equal to the previous one is skipped, which is how the three nodes of a CX·D·CX group collapse into one position on each of their two wires.

2. Pick the letter for this wire
A two-qubit unit (pair key ≥ 0) contributes label[0] if this wire is its first qubit, else label[1], and its position is stored in first_position or second_position accordingly. A one-qubit unit contributes its only letter; a wider or empty unit contributes '-'.

3. Extend or start a run
If the letter equals the previous position's letter, the run start is inherited; otherwise the run starts at this position. One linear scan, no nested loops.

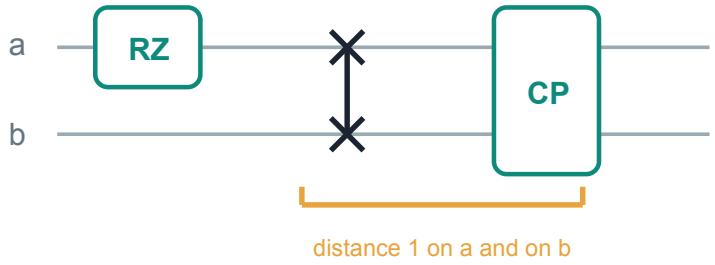
4. Three parallel lists per wire
sequences (unit ids), labels (one letter per position) and run_starts (one index per position) always have the same length, so a position indexes all three.

5. What this buys
'Do all units strictly between A and B carry letter P on this wire?' becomes two reads: the letter next to B and the run start there. commutes_between (later slide) is that check.

Stage 4a · reserve the neighbours a SWAP already has

`for swap in units.swaps:  look at positions ±1 on both wires  →  touching → taken,  or  pending.append(swap)`

A SWAP that already touches a same-pair unit on both wires

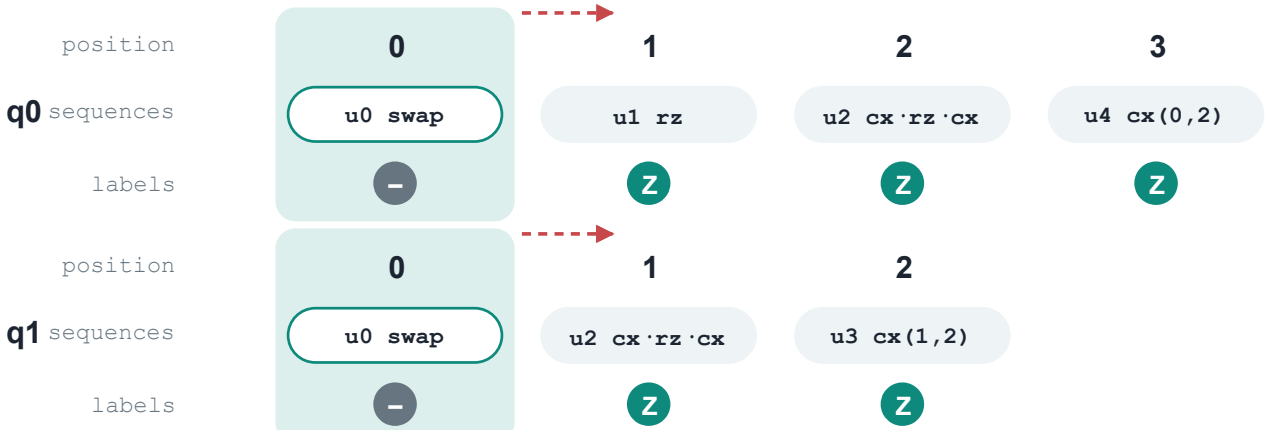


taken = {CP}: no other SWAP may claim this CP, and this SWAP is not added to pending: it already has its partner, and synthesis will merge the two whatever their letters.

Structural test only: pair key and positions, no letters. Adjacency must hold on both wires: a same-pair unit next door on one wire but farther away on the other is not a merge partner.

Runs over every SWAP before any search, including SWAPs the guard will skip later.

The running example: SWAP u0 at position 0 on q0 and on q1, and no neighbour qualifies



index −1 on q0: out of range

index 1 on q0: u1, pair_keys = −2 ≠ 1 → not a same-pair unit

touching = [] → pending = [u0]: on to the guard and the walk

index −1 on q1: out of range

index 1 on q1: u2, pair_keys = 1 ✓, but on q0 u2 sits at 2 and |2 − 0| ≠ 1

Reserving first protects existing partners and stops two SWAPs from claiming one unit: nothing in taken is selected in 4c.

Stage 4a · the reserve loop: how it works

```
def _select_moves(nodes: _NodeIndex, units: _UnitIndex, wires: _WireIndex):

    def position_on_wire(qubit, unit):
        if nodes.first_qubit[units.first_node[unit]] == qubit:
            return wires.first_position[unit]
        return wires.second_position[unit]

    ...  # commutes_between and splits_same_pair_run: see their own slides

    taken = set()
    pending = []
    for swap in units.swaps:
        pair_key = units.pair_keys[swap]
        head = units.first_node[swap]
        qubit_a, qubit_b = nodes.first_qubit[head], nodes.second_qubit[head]
        touching = []
        for qubit, at in ((qubit_a, wires.first_position[swap]),
                          (qubit_b, wires.second_position[swap])):
            sequence = wires.sequences[qubit]
            for index in (at - 1, at + 1):
                if (0 <= index < len(sequence)
                    and units.pair_keys[unit := sequence[index]] == pair_key
                    and abs(position_on_wire(qubit_a, unit)
                            - wires.first_position[swap]) == 1
                    and abs(position_on_wire(qubit_b, unit)
                            - wires.second_position[swap]) == 1):
                    touching.append(unit)
        if touching:
            taken.update(touching)
        else:
            pending.append(swap)
```

Step by step

1. position_on_wire
Given a wire and a unit, returns the unit's merged position on that wire: first_position if the wire is the unit's first qubit, else second_position. Every distance below is measured with it.

2. Every SWAP, in order
units.swaps lists only genuine standard swaps, numbered in topological order. For each one the loop fetches its pair key and its two qubits from its first node.

3. Four candidate slots
On wire a the positions at − 1 and at + 1, and the same on wire b. A slot is only considered when it is inside the wire.

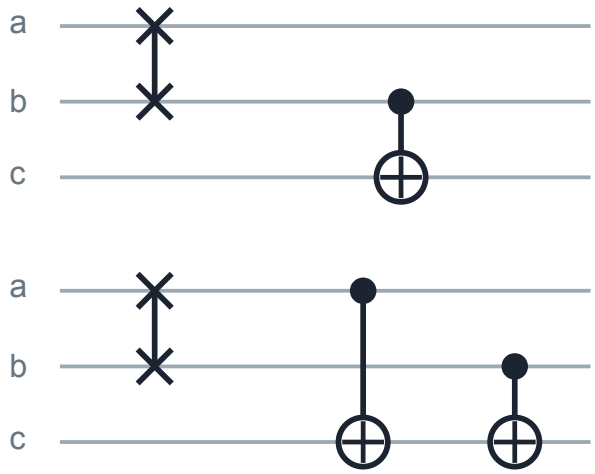
4. The touching test
The unit in the slot must have the SWAP's pair key, and its distance from the SWAP must be exactly 1 on wire a and on wire b. The second and third conditions catch a same-pair unit that is next door on one wire but not on the other.

5. taken or pending
Touching units are added to taken; a SWAP without any goes to pending. Only pending SWAPs reach the guard and the walk, and nothing in taken can be chosen later.

Stage 4b · _select_moves: the scheduling guard

```
if last_multi[qubit_a] <= first_position[head] or last_multi[qubit_b] <= second_position[head]: continue
```

Skip a pending SWAP when one of its wires has no later multi-qubit operation



no later multi-qubit op on a
→ skipped, nothing is searched

both wires continue with multi-qubit work
→ the walk runs for this SWAP

Running example

`head`	0	the SWAP's first node
`last_multi`	`[4, 4, 1]`	
`first_position[0]`	0	node position on q0
`second_position[0]`	0	node position on q1

4 > 0 on q0 and 4 > 0 on q1:
the SWAP survives the guard

Positions here are node positions from _NodeIndex, not the unit positions of _WireIndex.

Why: profit, not correctness

A merged SWAP + unit block ends in single-qubit gates. On a wire that is otherwise idle to the end of the circuit, an as-late-as-possible schedule would stretch that idle time, so such SWAPs are left alone.

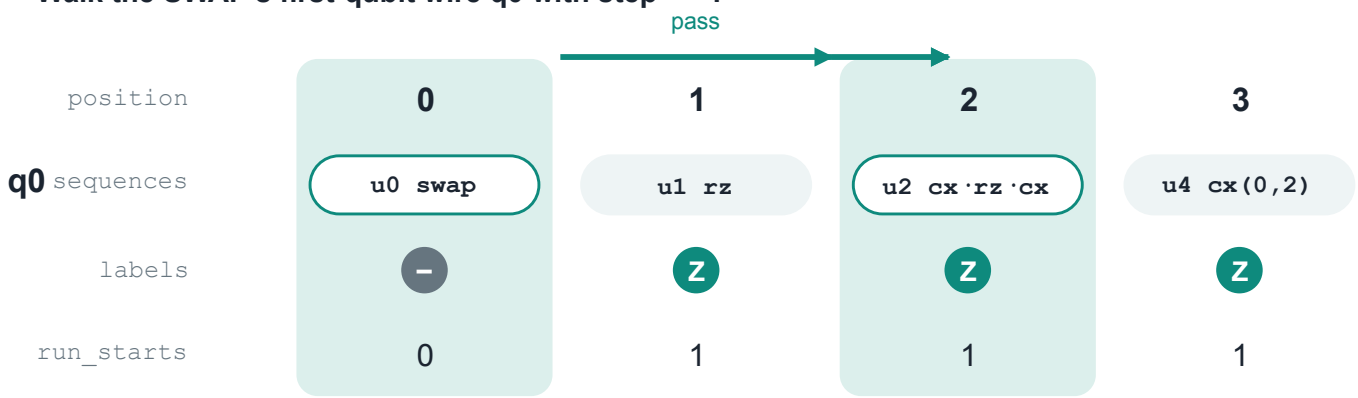
Details worth knowing

The guard reads the original circuit, before any move. A later same-pair interaction satisfies it on both wires. It runs only for pending SWAPs. A single-qubit gate after the SWAP does not count: last_multi ignores width-1 nodes.

Stage 4c · _select_moves: the walk, on the running example

`for step in (1, -1):  index = first_position[swap] + step;  walk;  check the stopping unit;  break on success`

Walk the SWAP's first-qubit wire q0 with step = +1



the loop:

```
while 0 <= index < size
  and pair_keys[sequence[index]] != pair_key
  and labels[index] != '-':  index += step
```

u1: pair −2 ≠ 1 and letter 'Z' ≠ '-' → keep walking

u2: pair_keys = 1 == the SWAP's → the walk stops here

The five checks on the stopping unit u2, with the running example's values

✓ `pair_keys[u2] == pair_key`	1 == 1: a same-pair unit, not a wall and not another pair's gate
✓ `u2 not in taken`	taken is empty: no SWAP reserved u2 in stage 4a
✓ `commutes_between(q0, u2, u0)`	positions 2 and 0: labels[1] == 'Z' and run_starts[1] = 1 <= 0 + 1
✓ `commutes_between(q1, u2, u0)`	positions 1 and 0: adjacent, nothing in between
✓ `not splits_same_pair_run(u2, +1, q0, q1)`	outward: u4 on q0, u3 on q1: different units, so no run is split

taken.add(u2); moves.append((2, 0, +1)); break. The step records that the unit was found after the SWAP; _apply_moves will place it right after it.

Stage 4c · the walk: how it works

```
moves = []
for swap in pending:
    head = units.first_node[swap]
    qubit_a, qubit_b = nodes.first_qubit[head], nodes.second_qubit[head]
    if (nodes.last_multi[qubit_a] <= nodes.first_position[head]
        or nodes.last_multi[qubit_b] <= nodes.second_position[head]):
        continue                                    # 4b: the scheduling guard
    pair_key = units.pair_keys[swap]
    sequence = wires.sequences[qubit_a]
    labels = wires.labels[qubit_a]
    size = len(sequence)
    # Search the SWAP's first-qubit wire, preferring a later candidate.
    for step in (1, -1):
        index = wires.first_position[swap] + step
        while (0 <= index < size
               and units.pair_keys[sequence[index]] != pair_key
               and labels[index] != "-"):
            index += step
        if not 0 <= index < size:
            continue
        unit = sequence[index]
        if (units.pair_keys[unit] == pair_key
            and unit not in taken
            and commutes_between(qubit_a, unit, swap)
            and commutes_between(qubit_b, unit, swap)
            and not splits_same_pair_run(unit, step, qubit_a, qubit_b)):
            taken.add(unit)
            moves.append((unit, swap, step))
            break
return moves
```

Step by step

1. Only pending SWAPs, after the guard
The loop body starts with the 4b condition; a SWAP that fails it is skipped with continue before any wire is read.

2. Walk wire a, later side first
index starts at the SWAP's position plus step. The while loop advances past units of other pairs whose letter is not '-'. It halts at a same-pair unit, at a wall ('-'), or past the end of the wire; in the last case the direction is given up.

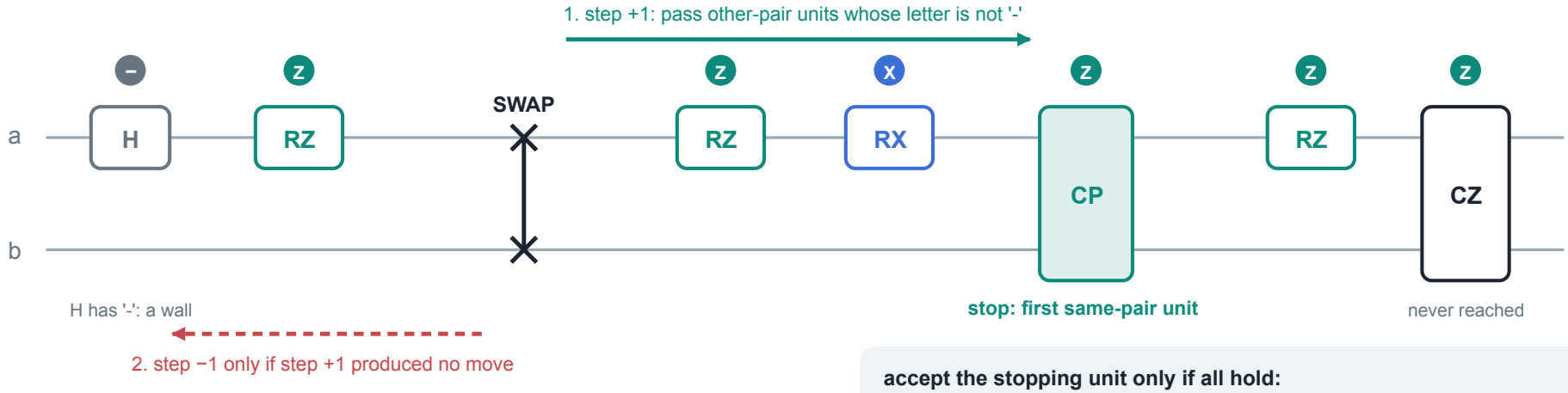
3. Five checks on the stopping unit
It must carry the SWAP's pair key (so a wall fails here), must not be in taken, must pass commutes_between on wire a and on wire b, and must not split a same-pair run. The walk never looks past a unit that fails.

4. Record and stop
On success the unit joins taken, (unit, swap, step) joins moves, and break leaves the direction loop: step = −1 is only tried when step = +1 produced no move.

5. What the result means
moves pairs each unit with its SWAP and the side it was found on. Candidates stay on their side of the SWAP, so a unit found after the SWAP is placed right after it and never crosses it.

Stage 4c · a walk that finds nothing

For a pending SWAP: walk wire a with step = +1, then step = −1



Step +1 stops at CP, but RX has letter X while CP has Z on wire a,
so commutes_between fails and CP is rejected on the spot.
Step −1 stops at H, whose letter is '-': not a same-pair unit, rejected.
The walk never looks past a stopping unit; this SWAP gets no partner.

accept the stopping unit only if all hold:

- same pair key as the SWAP
- not already taken
- `commutes_between on wire a`
- `commutes_between on wire b`
- `not splits_same_pair_run`

```
then: taken.add(unit); moves.append((unit, swap, step)); break
```

Why stop at the first same-pair unit

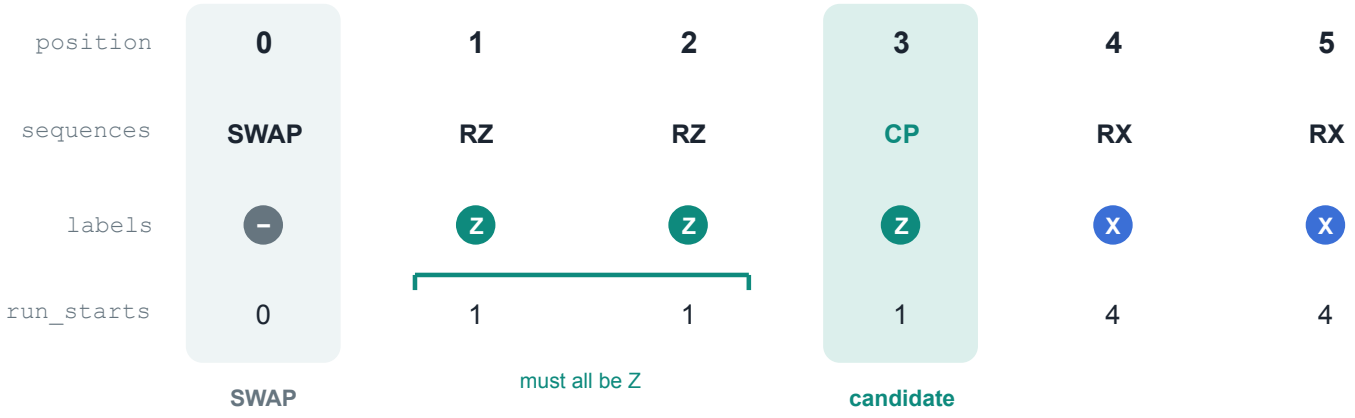
Moving CZ next to the SWAP would carry it across CP, a same-pair unit that cannot be reordered with it in general. The first same-pair unit is the only one that can reach the SWAP.

Why walls end the walk

A unit with letter '-' on wire a, such as H, a barrier or a measurement, cannot be crossed by anything, so no candidate beyond it could ever reach the SWAP.

commutes_between: the constant-time interval check

One wire of _WireIndex: positions, units, letters and where each run of equal letters starts



run_starts[i] = index where the run of equal letters containing position i began.

`Candidate at 3, SWAP at 0: look at position 2 only. labels[2] == 'Z' and run_starts[2] = 1 <= 0 + 1,`

so every unit strictly between them is Z → crossing allowed. No per-pair commutation check, whatever the distance.

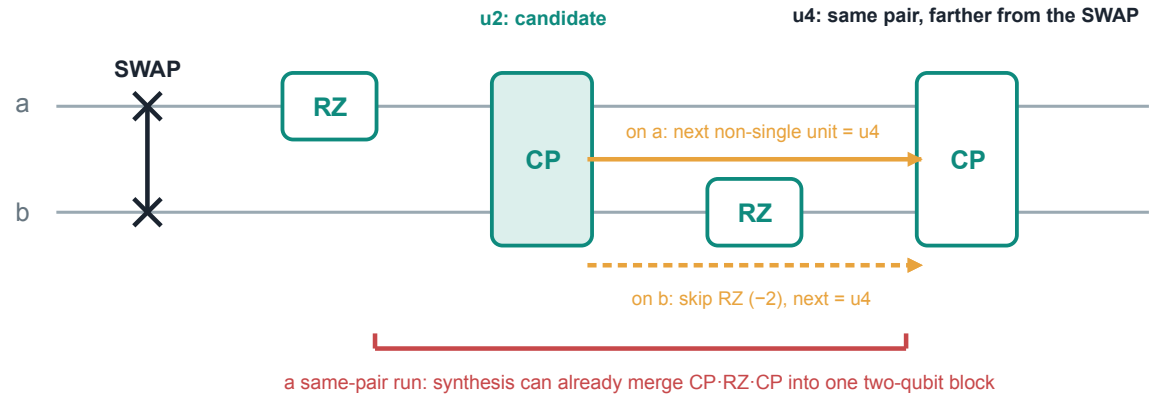
```
def commutes_between(qubit, unit, swap):
    unit_position = position_on_wire(qubit, unit)
    swap_position = position_on_wire(qubit, swap)
    low, high = ((unit_position, swap_position) if unit_position < swap_position
                 else (swap_position, unit_position))
    labels = wires.labels[qubit]
    pauli = labels[unit_position]
    if pauli == "-":
        return False
    return high - low <= 1 or (
        labels[high - 1] == pauli and wires.run_starts[qubit][high - 1] <= low + 1)
```

Why it is O(1)

Stage 3 precomputed run_starts, so the check reads two array entries whatever the distance. The walk is called once per SWAP and direction, and this check twice per stopping unit, so the search cost is dominated by the walk itself, not by commutation tests.

splits_same_pair_run: do not break up a run of same-pair work

SWAP; RZ(a); CP; RZ(b); CP on the same pair: the walk stops at the first CP, every letter check passes, and yet nothing moves



`splits_same_pair_run(u2, +1, a, b)`

step = +1: look away from the SWAP

`neighbours = [u4, u4]`

same unit on both wires, and

`pair_keys[u4] == pair_keys[u2]`

→ True: the move is refused;
moves = [] for this circuit

Pulling the first CP next to the SWAP would trade one mergeable neighbour for another and split a block synthesis could already fuse.

Without the second CP the same circuit yields moves = [(2, 0, +1)]. One-qubit units between the two are skipped, wider units stop the look-ahead.

```
def splits_same_pair_run(unit, step, qubit_a, qubit_b):
    neighbours = []
    for qubit in (qubit_a, qubit_b):
        sequence = wires.sequences[qubit]
        index = position_on_wire(qubit, unit) + step
        while (0 <= index < len(sequence)
               and units.pair_keys[sequence[index]] == _SINGLE_QUBIT_UNIT):
            index += step
        neighbours.append(sequence[index] if 0 <= index < len(sequence) else None)
    return (neighbours[0] is not None and neighbours[0] == neighbours[1]
            and units.pair_keys[neighbours[0]] == units.pair_keys[unit])
```

Reading the arguments

step is the walk direction, so unit + step is 'farther from the SWAP'. Only _SINGLE_QUBIT_UNIT (−2) units are skipped: a barrier (−1) or another pair's gate stops the look-ahead and the answer is False.

_apply_moves: rewriting the DAG

1 Remove the moving gates

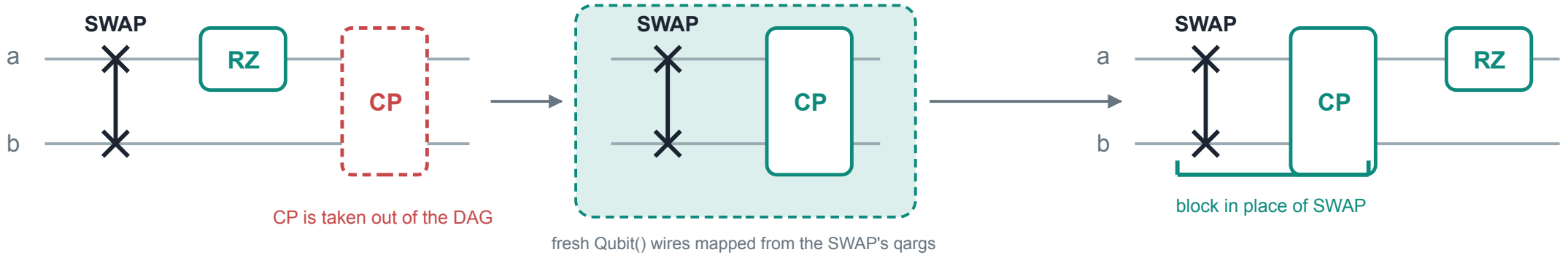
for every move, every member node

2 Build a tiny 2-qubit block

SWAP then unit (step +1) or unit then SWAP (step −1)

3 Substitute the block for the SWAP

`dag.substitute_node_with_dag(...)`



All removals happen before any insertion, so moves whose stretches overlap never interfere.

The original gate objects and parameters are reused unchanged; the DAG's global phase is kept; the same dag object is returned.

check=False on the block append is safe because the planner only admits quantum operations it already validated.

```
for unit, _, _ in moves:                        # 1. remove all members first
    for member in units[unit][0]: dag.remove_op_node(order[member])
for unit, swap, step in moves:                  # 2. build a 2-qubit block in the right order
    block = DAGCircuit(); block.add_qubits(qubits := [Qubit(), Qubit()]); wire_map = dict(zip(swap_node.qargs, qubits))
    ...; dag.substitute_node_with_dag(swap_node, block, wires=qubits)   # 3. replace the SWAP node by the block
```

The three index records

_NodeIndex	per node, from stage 1
`wires`	per qubit: node indices on that wire, in order
`last_multi`	per qubit: position of the last node with ≥ 2 qubits, or −1
`first_qubit`	index of the node's first qubit, or −1
`second_qubit`	index of its second qubit (two-qubit nodes only)
`first_position`	its index in wires[first_qubit]
`second_position`	its index in wires[second_qubit]
`widths`	number of qubits

_UnitIndex	per unit, from stage 2
`node_to_unit`	per node: its unit id
`first_node`	node index of the unit's first node
`groups`	unit → [first, middle, last], CX·D·CX units only
`labels`	letters, in the first node's qubit order
`pair_keys`	unordered pair key, or −2 (one-qubit) / −1 (other)
`swaps`	unit ids of genuine standard SWAPs, in order

_WireIndex	per wire or unit, from stage 3
`sequences`	per qubit: unit ids along the wire, repeats merged
`labels`	per qubit: the letter each unit carries on this wire
`run_starts`	per qubit and position: where the run of equal letters began
`first_position`	per unit: position on the wire of its first qubit, or −1
`second_position`	per unit: position on the wire of its second qubit

How to read an index

- A node index refers to order; a unit index refers to first_node; a qubit index refers to dag.qubits. A position is an index into one wire's list.
- _NodeIndex positions count nodes on a wire (the scheduling guard uses them). _WireIndex positions count merged units (every other stage-4 check uses them).
- frozen=True makes the records read-only after construction; slots=True keeps them compact. Nothing downstream mutates what an earlier stage built.

_Members, the helper functions and the performance picture

_Members	Role and source
`first`	unit id → index in order of its first node (units.first_node)
`groups`	unit id → the three node indices of a CX·D·CX unit (units.groups)
`q0, q1`	node index → first / second qubit index, or −1 (nodes.first_qubit, nodes.second_qubit)
`qargs`	callback returning all qubit indices of a node (qubits_for_node)
`__len__()`	number of units
`__getitem__(unit)`	(members, qubits): node indices and a frozenset of qubit indices

Helper	Lives in	Purpose
`qubits_for_node(node)`	`_find_moves`	all qubit indices of order[node]; passed to _Members as qargs
`next_on_wire(qubit, position)`	`_group_nodes`	node after that position on the wire, or None; used by the CX·D·CX test
`position_on_wire(qubit, unit)`	`_select_moves`	the unit's merged position on that wire (first_position or second_position)
`commutes_between(qubit, unit, swap)`	`_select_moves`	constant-time check that every unit between them carries the candidate's letter
`splits_same_pair_run(unit, step, qubit_a, qubit_b)`	`_select_moves`	True when moving would detach the unit from a same-pair unit beyond it

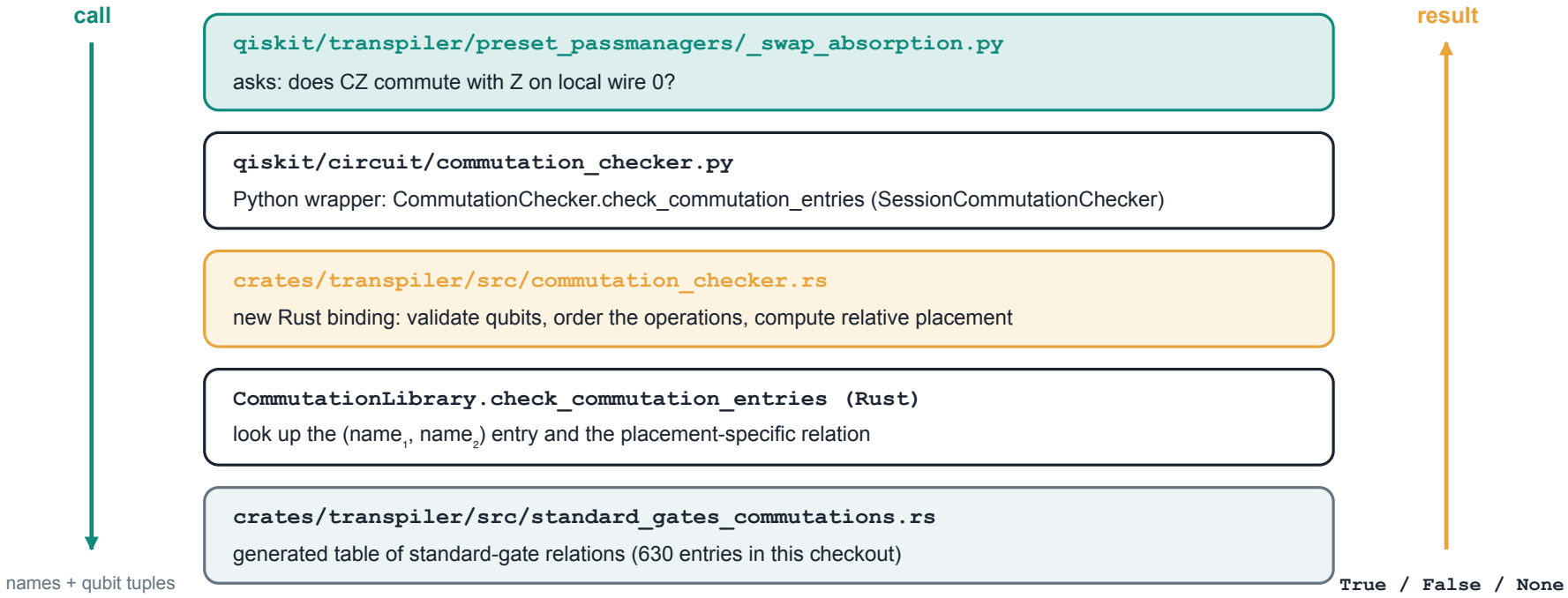
Performance picture

- Stages 1 to 3 are linear in the number of nodes and qubit incidences. commutes_between is O(1) thanks to run_starts; the walk and splits_same_pair_run still scan wire segments.
- Letters are looked up once per (name, width) per run: covering all 27 families costs 14×3 + 13×6 = 120 raw probes, regardless of how many gates or angles the circuit has.
- Exactly diagonal custom unitaries take the matrix-inspection branch and are not cached by name.

Part 4

The CommutationChecker change

What the CommutationChecker is



Three Python APIs on the class

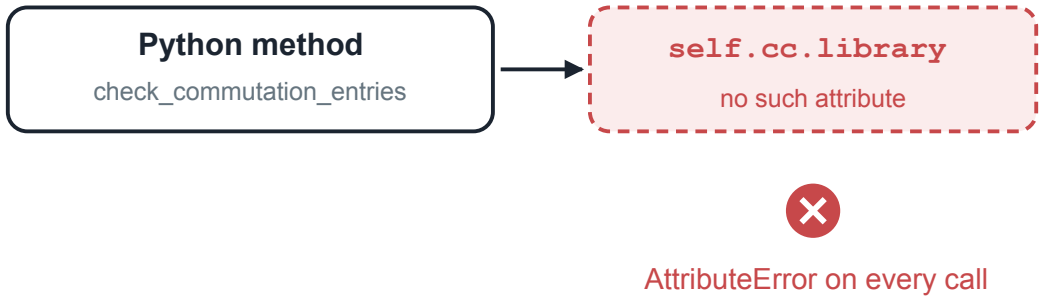
commute(…) → bool and commute_nodes(…) → bool are the general decisions, unchanged. check_commutation_entries(…) → bool | None is the raw table read, repaired here.

The instance the pass uses

SessionCommutationChecker (commutation_library.py) wraps get_standard_commutation_checker(), so its table is populated. A bare CommutationChecker() has an empty table and answers None.

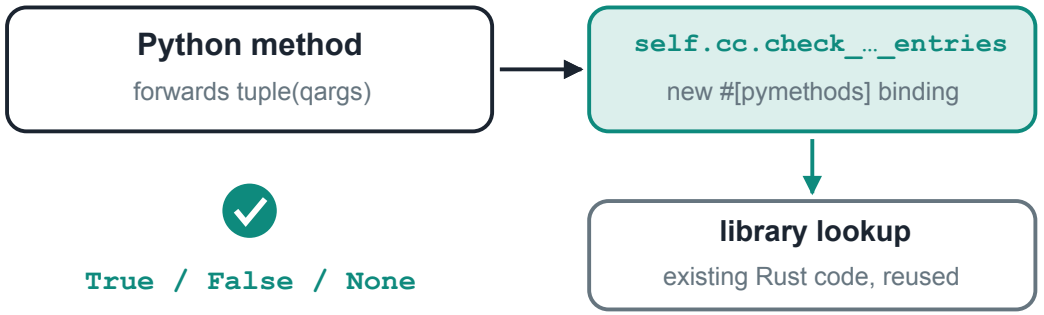
The defect and the repair

BEFORE: broken



The Rust object behind self.cc never exposed its library to Python, so the table was unreachable.

AFTER: commit c81cf602c



Same public name and arguments. The only Python change is which Rust method the wrapper calls.

Six files changed: the Python wrapper, the Rust binding, _swap_absorption.py, two test modules and a release note. The table itself is untouched.

```
# before
return self.cc.library.check_commutation_entries(
    first_op, first_qargs, second_op, second_qargs)
# AttributeError: no attribute 'library'
```

```
# after
return self.cc.check_commutation_entries(
    first_op, tuple(first_qargs),
    second_op, tuple(second_qargs))
```

Inside the new Rust binding

1 Check widths

X declares 1 qubit, CX declares 2.

`qargs (1,) and (0, 1) match → ok`

wrong length → QiskitError

2 Convert qubits

Both tuples go through one shared registry, so the same label on both sides becomes the same internal qubit

3 Reject duplicates

(a, a) inside one operation → QiskitError. Sharing qubits between the two operations is fine and needed

4 Disjoint → None

X on 2 versus CX on (0, 1): no shared qubit → return None. Compressed table entries describe overlapping placements only

5 Canonical order

sort by (num_qubits, len(name), name):

`('x', 1) comes before ('cx', 2);`

each op keeps its own qargs

6 Placement → lookup

X's qubit 1 is CX's 2nd qubit → `placement (1,). Table ('x','cx') at (1,)` → True. X on CX's control → False

`check_commutation_entries(XGate(), [1], CXGate(), [0, 1]) → True` · `X on [0] → False` · `X on [2] → None`

No parameter is read, no matrix is built, no definition is unrolled: the binding only validates, orders and looks up.

Relative placement lists, for each qubit of the canonical first operation, its position in the second, or None. X(b) against CX(a,b) has placement (1,); X(a) has (0,).

The contract of check_commutation_entries

Result	Meaning
`True`	The table stores a positive relation for this name pair and placement.
`False`	The table stores a negative relation. For a parameterized family this does not rule out commutation at special angles.
`None`	No applicable entry, the table is empty, or the two operations share no qubit.
`QiskitError`	A qubit sequence has the wrong length for its operation, or repeats a qubit. Qubits shared between the operations are fine.

```
from qiskit.circuit.commutation_library import (
    SessionCommutationChecker as checker)
from qiskit.circuit.library import (
    CXGate, CPhaseGate, XGate, ZGate)
lookup = checker.check_commutation_entries

lookup(ZGate(), [0], CXGate(), [0, 1])          # True
lookup(XGate(), [1], CXGate(), [0, 1])          # True
lookup(XGate(), [0], CXGate(), [0, 1])          # False
lookup(XGate(), [2], CXGate(), [0, 1])          # None
lookup(ZGate(), [0], CPhaseGate(0.3), [0, 1])   # None
```

What the raw lookup deliberately does not do

- It never reads parameter values, never maps rotations to generators, never applies the checker's gates filter, never unrolls a definition and never builds a matrix.
- It does not authenticate names: a custom gate called 'x' gets the standard 'x' entry. That is why the swap pass checks is_standard_gate() first.
- Explicitly supplied tables must use the canonical key order; reversed keys are not searched. Calling with the operations reversed is fine because inputs are canonicalized.

Why not just call commute()?

Question	commute(...)	check_commutation_entries(...)
Reads parameter values?	Yes: rotation normalization, near-identity acceptance	Never
Uses matrices or tolerances?	Yes, with a 1e-12 floor and matrix_max_num_qubits	Never
Applies the gates filter?	Yes	No
Result	bool; False may mean 'not commuting' or 'not checked'	True / False / None, as stored in the table
Disjoint qubits	Commute	None: compressed entries describe overlaps only
RY(1e-6) versus Z	May be accepted as identity	False for the whole family: the move is blocked

The swap pass only trusts 'is True'

Both False and None simply mean 'no letter for this axis'. Zero or several positive axes on a wire give '-'. No approximate fallback exists.

Not a universal exactness oracle

The table is generated numerically. Exactness here rests on the 27 admitted families and their algebraic identities, not on the table alone.

Files changed and how they are tested

File	Change
`qiskit/circuit/commutation_checker.py`	Repaired forwarding, Sequence[Qubit | int] annotations, documented contract
`crates/transpiler/src/commutation_checker.rs`	New Python-callable binding: validation, disjoint handling, canonical order, library lookup
`qiskit/transpiler/preset_passmanagers/_swap_absorption.py`	Hard-coded _WIRE_PAULIS table replaced by allowlist + probes + representatives + per-run cache; later refactored (ab28929ff) into four stage functions and three index records, behaviour unchanged
`test/python/circuit/test_commutation_checker.py`	TestCommutationTableLookup: placements, missing/disjoint, no fallback, invalid qubits, custom tables, no rotation normalization
`test/python/transpiler/test_swap_absorption.py`	Labels, exactness, unsupported operations, per-run lookup cache, movement, preset integration
`releasenotes/notes/fix-commutation-table-lookup-….yaml`	Release note for the repaired API

```
PYTHONPATH=. python -m unittest \
    test.python.circuit.test_commutation_checker \
    test.python.transpiler.test_swap_absorption
```

Tests worth opening first

- test_standard_wire_labels: the 27 letters for every angle and qubit order.
- test_lookup_cache_per_run: lookups happen only in preprocessing and never via commute().

Summary

1 What

_AbsorbIntoSwaps moves one two-qubit gate (or a CX-D-CX block) next to a routing SWAP on the same pair, so later synthesis compiles both as one block. SWAP + CX becomes 2 CX instead of 4.

2 Where

First pass of the translation stage at optimization levels 2 and 3. Plans with _find_moves in four stages (_index_nodes, _group_nodes, _index_unit_wires, _select_moves), rewrites with _apply_moves, never crosses a SWAP, never enters control flow.

3 Why it is safe

One Pauli letter per wire. Equal letters on every shared wire give exact commutation, global phase included. 27 standard families, diagonal unitaries, the CX-D-CX block.

4 What changed in the checker

check_commutation_entries used to raise AttributeError. A new Rust binding validates, canonicalizes and reads the configured table, returning True, False or None. The swap pass derives its letters from it through parameter-free representatives, once per gate kind per run.

5 Known limits

Greedy, one new partner per SWAP, no global search. Some commuting pairs (RXX vs RYY) are refused. Angles never inspected, so RY(0) still blocks. No guarantee a chosen move reduces gate count; the scheduling guard only avoids the obvious losses.

Where to look next

Source

`qiskit/transpiler/preset_passmanagers/_swap_absorption.py` the pass itself, about 510 lines after the readability refactor

`qiskit/transpiler/preset_passmanagers/common.py` generate_translation_passmanager() adds the pass at levels 2 and 3

`qiskit/circuit/commutation_checker.py` the Python wrapper and its documented contract

`crates/transpiler/src/commutation_checker.rs` the Rust binding check_commutation_entries

Tests and notes

`test/python/transpiler/test_swap_absorption.py`

`test/python/circuit/test_commutation_checker.py`

`design/swap_absorption_implementation_guide.md` the same material in prose

`design/old/Swap_Absorption_CommutationChecker_Plan.md` the design plan the change implemented

Background reading

Liu, Li, Zhou. Not All SWAPs Have the Same Cost: A Case for Optimization-Aware Qubit Routing. arXiv:2205.10596

Lao, Browne. 2QAN: A quantum compiler for 2-local qubit Hamiltonian simulation algorithms. arXiv:2108.02099

Deck rebuilt 26 October 2026 from commit ab28929ff (the _swap_absorption.py readability refactor). Diagrams are SVG files next to this deck.